

The Refined Learning Path

Phase 0 — Mental Model (3 days)

Goal: Know what you're building before you build it.

Read the README and architecture docs of exactly three systems:

- **Qdrant** — best documented internals
- **Weaviate** — good blog on their HNSW implementation choices
- **pgvector** — simplest, shows what "good enough" looks like

Don't read code yet. Read architecture docs and blog posts only. Answer one question after each: *what design trade-off did they make that the others didn't?*

Phase 1 — Why Vector Search Exists (4 days)

Goal: Understand the problem before the solution.

Read:

- *Introduction to Information Retrieval* — Manning et al. (free online)
 - Ch. 1, 2, 6, 7 only
 - ~4 hours total

Build: Inverted index + BM25 on 1000 documents. Make it actually search something. This is throwaway code — the point is understanding *why* keyword search breaks on semantic queries.

Key question to be able to answer: Why does BM25 fail to find "automobile" when you search "car"? What does a vector solve that an inverted index can't?

Phase 2 — Linear Algebra That Actually Matters (3 days)

Goal: Enough math to reason about distance functions, nothing more.

You don't need deep ML. You need:

- Dot product and what it geometrically means
- Vector norms (L1, L2)
- Cosine similarity — derive it from dot product + normalization
- Why normalized vectors make cosine = dot product (critical for performance)
- The curse of dimensionality — *why* distance functions break at high dimensions

Read:

- 3Blue1Brown's *Essence of Linear Algebra* — watch Ch. 1–4 only (YouTube, 2 hours)
- Then: the Wikipedia page on curse of dimensionality. Read it twice.

Key question: If I normalize all my vectors before storing them, can I use dot product instead of cosine similarity? What do I save computationally?

Phase 3 — Storage Engine Internals First (10 days)

This is the structural change from the ChatGPT roadmap. Do this before ANN algorithms.

Why: When you implement HNSW persistence later, you'll face: "how do I write this graph to disk without corrupting it on crash?" If you already know what a WAL is, this is a solved problem. If you don't, you'll invent something worse.

Read:

- *Designing Data-Intensive Applications* (DDIA) — Kleppmann
 - Ch. 1, 2, 3 — read fully (~6 hours)
 - Ch. 5 intro only (replication concepts, ~1 hour)
 - Skip Ch. 4, 6, 7, 8 for now

Then read:

- *Database Internals* — Alex Petrov
 - Ch. 2 (B-Trees) — read fully
 - Ch. 6 (Log-Structured Storage) — read fully
 - Skip rest for now

Key concepts you must be able to explain after this:

- What is a WAL and why does append-only prevent corruption?
- What is a page? What is a segment file?
- What is compaction and when does it happen?
- What's the difference between an LSM tree and a B+ tree? Which is better for write-heavy workloads?
- What does "crash recovery" actually mean at the byte level?

Build: A dead-simple append-only log in Python. Insert records. Kill the process. Replay the log on restart. This is 200 lines but it's the foundation of everything.

Phase 4 — ANN Algorithms (3 weeks)

Goal: Understand *why* HNSW, not just *how* to implement it.

Step 1 — Why brute force fails (2 days)

Implement brute force cosine search over 100k vectors. Measure QPS. This is your correctness baseline for everything that follows. Every index you build must produce recall $\geq 95\%$ of what this baseline finds.

Step 2 — The failed approaches (3 days)

Don't implement these, but understand *why* they fail:

- KD-Trees: read the Wikipedia page + this: at >20 dimensions, KD-tree degenerates to brute force. Understand why geometrically.
- Ball Trees: same failure mode, different structure.
- LSH (Locality Sensitive Hashing): read the original paper abstract. Understand the collision probability idea. Know why it was superseded.

Step 3 — IVF: Inverted File Index (3 days)

Read the FAISS paper: *"Billion-scale similarity search with GPUs"* — Johnson et al. (arXiv 2017). Read Section 3 only (IVF explanation). Then implement a basic IVF: k-means cluster your vectors, assign each vector to its nearest centroid, search only nearby clusters at query time.

This teaches the core ANN trade-off: you're sacrificing recall for speed by not searching everything.

Step 4 — Product Quantization (3 days)

Read: *"Product Quantization for Nearest Neighbor Search"* — Jégou et al. This is the compression backbone of FAISS. Understand: you split a 128-dim vector into 8 sub-vectors of 16 dims each, quantize each sub-space independently. This reduces 512 bytes per vector to ~ 8 bytes. Know the recall vs. memory trade-off this creates.

Step 5 — HNSW (1 week)

Read the paper: *"Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs"* — Malkov & Yashunin (2018).

Read it in this order:

1. Abstract + Introduction (understand the problem statement)
2. Section 4 (the algorithm) — read three times
3. Figure 1 (the graph structure) — stare at it until you can redraw it from memory
4. Section 5 (complexity analysis) — understand why $\log(n)$ search works
5. Skip Section 6 (experimental) for now

The four things you must be able to explain:

- Why does HNSW have multiple layers? What does each layer represent?
- How is the entry point chosen? What happens when it becomes stale?
- What is `ef_construction` vs `M` vs `ef_search`? What does each control?
- Why does greedy search on the graph find approximate neighbors? When does it fail?

Phase 5 — Read FAISS Source (4 days)

Goal: See how a production system structures these ideas.

Don't read the whole codebase. Read in this order:

1. faiss/Index.h — the base interface (30 min)
2. faiss/IndexFlat.cpp — brute force baseline (1 hour)
3. faiss/IndexIVF.cpp — IVF implementation (2 hours)
4. faiss/IndexHNSW.cpp — HNSW implementation (3 hours)

For each file, answer: What data structure holds the index? How is search implemented? How is add/insert implemented? What's the memory layout?

Then read the FAISS wiki page on "*Guidelines to choose an index*". This is the best single-page summary of ANN trade-offs that exists.

Phase 6 — Memory, Cache, and Performance Mental Model (4 days)

This is the gap the ChatGPT roadmap misses entirely.

You cannot build a fast vector index without understanding why memory access patterns matter.

Read:

- "*What Every Programmer Should Know About Memory*" — Ulrich Drepper (free PDF). Read Parts 1, 2, and 6 only (~3 hours)
- Brendan Gregg's blog post: "*CPU Flame Graphs*" (30 min)

Key concepts:

- L1/L2/L3 cache sizes and latency numbers (memorize: L1 ~1ns, RAM ~100ns)
- Cache line size (64 bytes) and why struct layout matters for HNSW neighbor lists
- Why random access patterns (like graph traversal in HNSW) are cache-unfriendly
- What memory bandwidth bound vs. compute bound means

Key question: Your HNSW neighbor list stores node IDs as int32. Each node has M=16 neighbors. That's 64 bytes per node's neighbor list — exactly one cache line. Is that intentional? (It is in production implementations.)

Phase 7 — Build the Vector Database (6–8 weeks with AI assistance)

Now you use OpenCode/Codex/Claude aggressively. But you direct them with architectural intent.

Week 1–2: Storage Layer

- Append-only segment files for vectors (binary format, fixed-width records)

- WAL for all mutations
- Metadata store (SQLite is fine for now)
- Crash recovery tests: kill -9 during insert, verify recovery
- *You write this. The AI helps with boilerplate. You own the crash recovery logic.*

Week 3–4: HNSW Core

- In-memory HNSW: node struct, layer assignment, greedy search, neighbor selection (heuristic, not simple)
- Benchmark recall@10 against brute force on GloVe-25 dataset
- Parameter sweep: $M \in \{8, 16, 32\}$, $ef_construction \in \{100, 200, 400\}$
- *Use AI to generate node struct and basic search. You write the neighbor selection heuristic and benchmark harness.*

Week 5: Persistence

- Serialize HNSW graph to disk (this is non-trivial: the graph has pointer-based edges that need to become index-based)
- WAL integration: how do you replay inserts into HNSW without rebuilding from scratch?
- Crash recovery for the graph
- *This is where your Phase 3 knowledge pays off. The AI will suggest wrong approaches. You'll know why.*

Week 6: Query Engine

- Metadata filtering: pre-filter vs. post-filter decision logic
- Hybrid search: BM25 + vector (reciprocal rank fusion)
- Top-K with filter pushdown

Week 7: Benchmarking and Observability

- ann-benchmarks dataset (SIFT-1M or GloVe)
- Recall@10, P50/P95/P99 latency, QPS under concurrent load
- Prometheus metrics endpoint
- Memory usage profiling

Week 8: Polish

- pgvector-compatible API subset (so existing clients can connect)
 - README with architecture diagram + benchmark results
 - One blog post: "What I learned building a vector database from scratch"
-

The Complete Reading List, Sequenced

Must-read, in order:

1. *Introduction to Information Retrieval* — Manning (Ch. 1, 2, 6, 7)
2. *Designing Data-Intensive Applications* — Kleppmann (Ch. 1, 2, 3, 5 intro)
3. *Database Internals* — Petrov (Ch. 2, 6)
4. FAISS paper — Johnson et al. (Section 3 only)
5. Product Quantization paper — Jégou et al. (full)
6. HNSW paper — Malkov & Yashunin (full, read Section 4 three times)
7. *What Every Programmer Should Know About Memory* — Drepper (Parts 1, 2, 6)

Reference (consult when relevant, don't read front-to-back):

- *Systems Performance* — Brendan Gregg (Ch. 2 for methodology, rest on demand)
- FAISS wiki (guidelines to choose an index — read fully)
- Qdrant's blog (their HNSW implementation posts specifically)

Skip entirely:

- Any "build a vector database with LangChain" tutorial
- Any tutorial that starts with pip install chromadb
- The full text of DDIA Ch. 6, 7, 8 (save for if/when you build the distributed layer)

The One Principle That Governs All of This

When you use OpenCode or Codex to write a component, you should be able to:

1. Describe the interface before it generates it
2. Identify the bug in what it generates (there will be one)
3. Explain to a senior engineer why you made the architectural choices you made

If you can't do those three things for a component, you haven't learned enough about that component yet. Go back and read before generating.

The AI writes the code faster. You provide the judgment that makes the code correct. That division of labor is the actual skill.